

05 — Strategy Pattern & API Integration

What problem are we solving?

Our ETL pipeline needs to call external AI services — for translating text and generating embeddings. But here's the catch: we want to support **multiple providers** (Gemini, Groq, OpenRouter) and let the user switch between them from a dropdown in the Admin UI, without touching any code.

If we hardcoded Gemini API calls everywhere, switching to Groq would mean finding and rewriting every call. That's fragile and error-prone.

The Strategy Pattern — explained simply

Imagine you're ordering food. You don't care whether the restaurant uses a gas oven, wood fire, or microwave — you just say "make me a pizza" and get a pizza back. The **strategy** is HOW the pizza gets made, but your order (the interface) stays the same.

In code terms:

```
You (caller) → "generate text please" → [Strategy picks: Gemini? Groq? OpenRouter?] → Result
```

The Strategy Pattern says: **define a common interface, then let different implementations fulfill it.** The caller never knows or cares which implementation is running.

How we built it

Step 1: Define the interface (Abstract Base Class)

```
from abc import ABC, abstractmethod

class LLMProvider(ABC):
    name: str = "base"

    @abstractmethod
    def generate(self, prompt: str, system_prompt: str = "") -> str:
        ...
```

This is our "contract" — every LLM provider MUST have a `generate()` method that takes a prompt and returns text. The `ABC` (Abstract Base Class) enforces this — if someone creates a new provider and forgets `generate()`, Python raises an error immediately.

Step 2: Implement concrete strategies

Each provider implements the same interface but talks to a different API:

```
class GeminiLLM(LLMProvider):
    name = "gemini"
```

```

def generate(self, prompt, system_prompt=""):
    # Calls Google's Gemini API
    # Returns the generated text

class GroqLLM(LLMProvider):
    name = "groq"

    def generate(self, prompt, system_prompt=""):
        # Calls Groq's API (OpenAI-compatible format)
        # Returns the generated text

```

Step 3: A factory function picks the right one

```

PROVIDERS = {
    "gemini": GeminiLLM,
    "groq": GroqLLM,
    "openrouter": OpenRouterLLM,
}

def get_llm_provider(provider_name=None):
    if provider_name is None:
        provider_name = _get_active_provider_from_db() # reads Admin UI setting

    cls = PROVIDERS[provider_name]
    return cls()

```

The rest of the code just calls `get_llm_provider()` — it doesn't know or care which provider is active. When the admin changes the dropdown from "Gemini" to "Groq", the next ETL run automatically uses Groq.

API Integration patterns we used

Pattern 1: Google Gemini (custom REST API)

Gemini uses its own JSON format:

```

payload = {
    "contents": [
        {"role": "user", "parts": [{"text": "system instructions here"}]},
        {"role": "model", "parts": [{"text": "Understood."}]},
        {"role": "user", "parts": [{"text": "actual prompt here"}]}
    ]
}

```

Gemini doesn't have a native "system" role, so we simulate it by putting the system prompt as the first user message and having the model "acknowledge" it. This is a common workaround.

Pattern 2: OpenAI-compatible APIs (Groq, OpenRouter)

Both Groq and OpenRouter use the same format as OpenAI — this is intentional. Many AI providers adopt OpenAI's API format so developers can switch easily:

```

payload = {
    "model": "llama-3.3-70b-versatile",
    "messages": [
        {"role": "system", "content": "system instructions"},
        {"role": "user", "content": "actual prompt"}
    ]
}

```

```
}
```

The only differences between Groq and OpenRouter are:

- The **URL** they send requests to
- The **model name** they specify
- The **API key** they use for authentication

Everything else is identical — which is why the OpenAI format has become a de facto standard.

Authentication patterns

All three providers use API keys, but in different ways:

Provider	Auth Method	Example
Gemini	Query parameter	?key=AIza... appended to URL
Groq	Bearer token header	Authorization: Bearer gsk_...
OpenRouter	Bearer token header	Authorization: Bearer sk-or-...

Bearer tokens in headers are generally more secure than query parameters (URLs can end up in server logs), but Gemini's approach works fine for development.

Why this matters for our project

1. **R22 requires multi-provider LLM support** — the Strategy Pattern satisfies this cleanly
2. **R24 requires runtime configuration** — reading the active provider from the database (set via Admin UI) means zero code changes to switch
3. **UAT evidence** — every API call logs `provider=gemini model=gemini-2.5-flash`, making it easy to prove which provider served each operation

Key vocabulary

Term	Meaning
Strategy Pattern	A design pattern where you define a family of algorithms (strategies), encapsulate each one, and make them interchangeable at runtime
Abstract Base Class (ABC)	A Python class that defines methods subclasses MUST implement — acts as a contract
Factory function	A function that creates and returns objects — the caller doesn't need to know the exact class
REST API	An API accessed over HTTP using standard methods (GET, POST) with JSON payloads

Term	Meaning
Bearer token	An authentication method where you include <code>Authorization: Bearer</code> in request headers
OpenAI-compatible	APIs that accept the same JSON format as OpenAI's Chat Completions API

Test yourself

1. If we wanted to add a new LLM provider (say, Anthropic Claude), what three things would we need to do?
2. Why do we read the active provider from the database instead of from a config file?
3. What's the advantage of Groq and OpenRouter both using OpenAI-compatible formats?
4. Why does the `generate()` method strip whitespace from the result before returning?